

# Experimentação — Testes Automatizados do Backend (Agende Aqui)

Gabriel Linard Leite

2026

## Testes Automatizados dos Módulos Centrais

Esta seção documenta a suíte de testes automatizados construída para validar os três módulos centrais da plataforma Agende Aqui — Urgências Não Emergenciais, Formulários Pré-Consulta e Sistema de Reaproveitamento de Horários — e os mecanismos transversais de autenticação, controle de acesso e proteção de dados descritos na Seção IV do artigo. Cada teste é apresentado de forma declarativa: qual pergunta do projeto ele responde, e qual foi o resultado obtido ao executá-lo contra o código real do backend.

### A. Objetivos do experimento

1. Verificar se as regras de prazo do Sistema de Reaproveitamento de Horários (janela de 48h para cancelamento, janela de confirmação de presença entre 48h e 15h, bloqueio de 60 dias por ausência pós-confirmação) são aplicadas pelo servidor, e não apenas ocultadas na interface.
2. Verificar se a lógica de desempate do Sistema de Reaproveitamento de Horários respeita a prioridade definida (urgência primeiro, depois proximidade da consulta) mesmo sob concorrência real.
3. Verificar se as validações de cadastro (CPF, formato e unicidade do registro em conselho profissional) impedem perfis inválidos ou duplicados.
4. Verificar se as rotinas assíncronas de lembrete e notificação continuam funcionando de forma consistente mesmo quando uma etapa individual (envio de e-mail) falha.
5. Verificar se o controle de acesso por perfil impede que um usuário autenticado leia ou altere dados de outro usuário, e se as consultas ao banco estão protegidas contra injeção de SQL.

### B. Instrumentos e materiais

| Ferramenta                                | Função no experimento                                                                                                                     |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Jest 30                                   | Executor de testes e biblioteca de asserções                                                                                              |
| Supertest 7                               | Envio de requisições HTTP reais contra a aplicação Express, sem abrir uma porta de rede                                                   |
| Mock manual do driver <code>mysql2</code> | Substitui o banco de dados por um roteador de consultas controlado, simulando qualquer estado do banco sem precisar de uma instância real |

| Ferramenta                        | Função no experimento                                                                                                            |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Mock do módulo de e-mail (Resend) | Substitui o envio real de e-mail por funções espíãs, evitando I/O de rede e permitindo verificar quais e-mails seriam disparados |

Nenhum teste depende de uma instância real de MySQL ou de um provedor de e-mail — a suíte inteira roda de forma determinística e isolada.

## C. Resultado consolidado

A suíte foi executada com `npm test` dentro de `backend/`.

| Arquivo                                          | Categoria                     | Testes    | Resultado           |
|--------------------------------------------------|-------------------------------|-----------|---------------------|
| <code>auth.middleware.unit.test.js</code>        | Unitário                      | 5         | ✅ 5/5 aprovados     |
| <code>usuariosModel.bloqueio.unit.test.js</code> | Unitário                      | 4         | ✅ 4/4 aprovados     |
| <code>reservas.regras.test.js</code>             | Regra de negócio / Integração | 7         | ✅ 7/7 aprovados     |
| <code>register.regras.test.js</code>             | Regra de negócio / Integração | 16        | ✅ 16/16 aprovados   |
| <code>vagas.concorrencia.test.js</code>          | Concorrência                  | 3         | ✅ 3/3 aprovados     |
| <code>security.test.js</code>                    | Segurança                     | 13        | ✅ 13/13 aprovados   |
| <code>notificacoes.jobs.test.js</code>           | Unitário                      | 9         | ✅ 9/9 aprovados     |
| <code>idor-ampliado.test.js</code>               | Segurança                     | 20        | ✅ 20/20 aprovados   |
| <code>uploads.security.test.js</code>            | Segurança                     | 7         | ✅ 7/7 aprovados     |
| <code>seguranca-ampliada2.test.js</code>         | Segurança                     | 8         | ✅ 8/8 aprovados     |
| <code>auditoria.test.js</code>                   | Segurança                     | 5         | ✅ 5/5 aprovados     |
| <b>Total</b>                                     |                               | <b>97</b> | <b>97/97 (100%)</b> |

O arquivo `idor-ampliado.test.js` nasceu de uma segunda rodada de auditoria, mais ampla, motivada por uma pergunta direta sobre o quão seguro o sistema realmente estava. O arquivo `uploads.security.test.js` nasceu de uma terceira rodada, motivada por uma pergunta específica sobre vazamento de dados sensíveis (exames). O arquivo `seguranca-ampliada2.test.js` nasceu de uma quarta rodada, desta vez varrendo classes inteiras de vulnerabilidade (não só rotas específicas) — força bruta, segredo de sessão, exposição de dados por enumeração e o direito de exclusão de conta previsto na LGPD. O arquivo `auditoria.test.js` fecha o gap que havia ficado registrado como pendente na rodada anterior: nenhuma ação sensível (login, cadastro, redefinição de senha, exclusão de conta) ficava registrada em lugar nenhum — ver item F.

Tempo total de execução: aproximadamente 14 segundos, sem dependências externas.

## D. Procedimento

Os 97 testes foram organizados em cinco categorias metodológicas:

- **Unitário:** exercita uma função isolada (middleware, regra de modelo, job de background), com toda dependência externa substituída por mock.
- **Integração:** sobe a aplicação Express real e envia requisições HTTP via Supertest, exercitando roteamento e middlewares de ponta a ponta, com o banco mockado apenas na fronteira do driver.
- **Regra de negócio:** verifica, via API HTTP, que uma regra documentada no artigo é de fato aplicada pelo servidor (não apenas pela interface).
- **Segurança:** verifica autenticação, autorização e proteção contra injeção de SQL.
- **Concorrência:** dispara requisições HTTP simultâneas de verdade ( `Promise.all` ), sem simular tempo, para observar o comportamento real sob disputa.

## E. O que cada teste responde e qual foi o resultado

### E.1 Autenticação por token (unitário) — `auth.middleware.unit.test.js`

O artigo (Seção IV-A) declara que “rotas que expõem dados sensíveis (...) são protegidas por um *middleware* de autenticação que valida a assinatura e a expiração do token antes de processar a requisição, retornando HTTP 401 em caso de falha”. Estes 5 testes verificam essa afirmação na função `authenticate` isoladamente.

| Teste                                                  | Pergunta que responde                                                 | Resultado obtido                                                                                    |
|--------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Requisição sem cabeçalho<br><code>Authorization</code> | O middleware bloqueia quem não envia token nenhum?                    | <b>Sim</b> — retorna 401 e a mensagem “Não autenticado.”                                            |
| Cabeçalho sem o prefixo <code>Bearer</code>            | O middleware exige o formato correto do cabeçalho?                    | <b>Sim</b> — retorna 401                                                                            |
| Token assinado com segredo diferente do servidor       | Um token forjado por terceiros é aceito?                              | <b>Não</b> — retorna 401, a assinatura é validada de fato                                           |
| Token expirado                                         | Um token vencido continua valendo?                                    | <b>Não</b> — retorna 401, mesmo com o <code>id</code> correto no payload                            |
| Token válido                                           | Um token correto é aceito e o <code>id</code> do usuário é propagado? | <b>Sim</b> — <code>req.userId</code> é populado e a requisição segue ( <code>next()</code> chamado) |

**Resultado do arquivo: 5/5 aprovados.** A implementação do JWT confere com o que o artigo descreve.

### E.2 Bloqueio de 60 dias por ausência pós-confirmação (unitário) — `usuariosModel.bloqueio.unit.test.js`

O artigo (Seção IV-B) afirma: “o sistema registra o evento como ausência pós-confirmação e aplica um bloqueio temporário de 60 dias”. Estes 4 testes verificam a regra diretamente na camada de modelo.

| Teste                                 | Pergunta que responde                                    | Resultado obtido                                                                                                             |
|---------------------------------------|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>bloqueado_ate</code> nulo       | Um usuário que nunca faltou é tratado como bloqueado?    | <b>Não</b> — <code>getBloqueio</code> retorna <code>null</code>                                                              |
| <code>bloqueado_ate</code> no passado | Um bloqueio vencido continua impedindo o agendamento?    | <b>Não</b> — retorna <code>null</code> , o paciente recupera o acesso automaticamente, como o artigo promete                 |
| <code>bloqueado_ate</code> no futuro  | Um bloqueio vigente é detectado e o motivo é preservado? | <b>Sim</b> — retorna o registro completo com o motivo da ausência                                                            |
| Aplicação de um novo bloqueio         | O sistema usa exatamente 60 dias, e não outro valor?     | <b>Sim</b> — <code>bloquearTemporariamente</code> usa <code>BLOQUEIO_DIAS = 60</code> , confirmando o valor citado no artigo |

**Resultado do arquivo: 4/4 aprovados.**

### E.3 Janela de 48h e bloqueio ao agendar (regra de negócio + integração) — `reservas.regras.test.js`

O artigo (Seção IV-B) afirma: “o cancelamento está disponível apenas com antecedência mínima de 48 horas em relação ao horário da consulta. Essa restrição é aplicada tanto na interface (...) quanto no *back-end*, que recusa a requisição caso a condição não seja satisfeita, impedindo que a regra seja contornada”. Estes 7 testes atacam a API diretamente — sem passar pela interface — para verificar se essa promessa se sustenta.

| Teste                                                       | Pergunta que responde                                                                                    | Resultado obtido                                                                                         |
|-------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Cancelamento a 10h da consulta                              | Um cliente que ignora a interface e chama a API direto consegue cancelar dentro das 48h?                 | <b>Não</b> — HTTP 403, mensagem cita a janela de 48h. A regra realmente está no servidor, não só na tela |
| Cancelamento a 72h da consulta                              | Fora da janela protegida, o cancelamento funciona normalmente?                                           | <b>Sim</b> — HTTP 200                                                                                    |
| Cancelamento a exatamente 48h                               | No limite exato do prazo, o sistema erra para o lado mais seguro (bloqueia) ou mais permissivo (libera)? | <b>Bloqueia</b> — HTTP 403, comportamento protetivo no limite                                            |
| Profissional exclui a reserva de um paciente dentro das 48h | A regra de 48h também impede o profissional de agir, ou é exclusiva do paciente?                         | <b>É exclusiva do paciente</b> — HTTP 200, o profissional pode remover a reserva mesmo dentro da janela  |
| Exclusão de reserva inexistente                             | O servidor lida corretamente com um id que não existe?                                                   | <b>Sim</b> — HTTP 404                                                                                    |
| Paciente bloqueado tenta criar reserva                      | O bloqueio por ausência realmente impede novos agendamentos?                                             | <b>Sim</b> — HTTP 403 com o motivo do bloqueio na mensagem                                               |
| Paciente com bloqueio expirado tenta criar reserva          | O paciente recupera o acesso automaticamente após os 60 dias?                                            | <b>Sim</b> — HTTP 200, agendamento aceito normalmente                                                    |

**Resultado do arquivo: 7/7 aprovados.** A promessa do artigo de que a regra “não pode ser contornada” pela interface se confirma: o servidor recusa a requisição independentemente de o cliente ser o site oficial ou uma chamada direta à API.

#### E.4 Validação de cadastro — CPF e registro em conselho (regra de negócio + integração) — `register.regras.test.js`

O artigo (Seção II-G) afirma que a plataforma “adota esse controle ao exigir, no cadastro, o número de registro no conselho correspondente à especialidade declarada (...), aceitando apenas um registro por profissional, o que impede perfis duplicados”. Estes 16 testes verificam, campo a campo, se essas garantias realmente existem no `POST /register`. Como o arquivo dispara 16 requisições `POST /register` reaproveitando a mesma instância da aplicação (mesmo IP simulado em todas as chamadas), o `registerLimiter` — que também existe e é aplicado normalmente em produção, com o mesmo mecanismo do `loginLimiter` verificado na Seção E.10 (15 tentativas/hora) — é mockado aqui: este arquivo testa regras de negócio de cadastro, não o rate limit em si, para não acoplar dois tipos diferentes de garantia num único conjunto de testes.

| Teste                                            | Pergunta que responde                                                                                                                                                                              | Resultado obtido                                                                                                        |
|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| CPF com dígito verificador inválido              | O sistema calcula o dígito verificador de verdade, ou só confere o tamanho?                                                                                                                        | <b>Calcula de verdade</b> — HTTP 400                                                                                    |
| CPF com todos os dígitos iguais                  | Um CPF “fácil” tipo 111.111.111-11 é aceito?                                                                                                                                                       | <b>Não</b> — HTTP 400, caso tratado explicitamente                                                                      |
| CPF com quantidade de dígitos errada             | O sistema valida o tamanho antes do checksum?                                                                                                                                                      | <b>Sim</b> — HTTP 400 com mensagem específica                                                                           |
| CPF com dígito verificador correto               | Um CPF genuinamente válido passa pela validação?                                                                                                                                                   | <b>Sim</b> — a validação de CPF não bloqueia o cadastro                                                                 |
| Gênero fora da lista aceita                      | Valores arbitrários no campo gênero são aceitos?                                                                                                                                                   | <b>Não</b> — HTTP 400                                                                                                   |
| E-mail já cadastrado                             | O sistema impede duas contas com o mesmo e-mail?                                                                                                                                                   | <b>Sim</b> — HTTP 409                                                                                                   |
| CPF já cadastrado, sem elegibilidade de upgrade  | Um CPF duplicado sempre é bloqueado, mesmo quando não se trata de um paciente virando profissional?                                                                                                | <b>Sim</b> — HTTP 409                                                                                                   |
| Cadastro do mesmo paciente processado duas vezes | A rotina de dados de exemplo que roda uma vez após o cadastro (usada para já popular a tela do usuário recém-criado) cria registros duplicados se disparada mais de uma vez para o mesmo paciente? | <b>Não</b> — nenhuma segunda reserva é inserida quando já existe uma reserva de exemplo confirmada para aquele paciente |
| Profissional sem <code>tipoProfissional</code>   | O sistema exige que o tipo de profissão seja informado?                                                                                                                                            | <b>Sim</b> — HTTP 400                                                                                                   |

| Teste                                           | Pergunta que responde                                                           | Resultado obtido                                                            |
|-------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Médico sem especialidade                        | A obrigatoriedade da especialidade médica é aplicada no servidor?               | <b>Sim</b> — HTTP 400                                                       |
| Profissional “outros” sem profissão customizada | O caso especial “outros” exige o campo livre?                                   | <b>Sim</b> — HTTP 400                                                       |
| Profissional sem número de conselho             | O número de registro no conselho é realmente obrigatório?                       | <b>Sim</b> — HTTP 400, confirmando o texto do artigo                        |
| CRM fora do formato CRM/UF NNNNNN               | O formato do número de conselho é validado por categoria, ou aceito livremente? | <b>É validado</b> — HTTP 400 com o formato esperado na mensagem             |
| CREFITO fora do formato CREDITO-N/NNNNNN-F      | O mesmo vale para fisioterapeutas, com um padrão totalmente diferente do CRM?   | <b>Sim</b> — HTTP 400, confirmando que cada categoria tem sua própria regra |
| Número de conselho válido, mas já cadastrado    | “Um registro por profissional” é aplicado mesmo com o formato correto?          | <b>Sim</b> — HTTP 409, perfil duplicado é impedido                          |
| Profissional sem UF/Região                      | A UF do conselho é obrigatória, como a Tabela 26 da ANS pressupõe?              | <b>Sim</b> — HTTP 400                                                       |

**Resultado do arquivo: 16/16 aprovados.** Todas as garantias de unicidade e formato descritas na Seção II-G do artigo se confirmam na prática.

## E.5 Desempate concorrente do Sistema de Reaproveitamento de Horários (concorrência) — `vagas.concorrencia.test.js`

O artigo (Seção IV-B) descreve o mecanismo de desempate: “o sistema aplica uma janela de desempate de aproximadamente 1,2 segundo (...). Em caso de empate, a prioridade segue dois critérios em ordem: primeiro, pacientes com solicitação de urgência não emergencial têm precedência sobre os demais; segundo, (...) vence aquele cuja consulta estava mais próxima em data e horário da vaga liberada”. Estes 3 testes disparam requisições HTTP **simultâneas de verdade** (não simuladas) para verificar se essa prioridade realmente se sustenta sob concorrência real.

| Teste                                                                       | Pergunta que responde                                                                                                                 | Resultado obtido                                                                                                             |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Candidato urgente concorre com um não urgente que estava mais perto da vaga | A urgência realmente tem prioridade sobre a proximidade, mesmo quando isso é “contraintuitivo” (o outro candidato estava mais perto)? | <b>Sim</b> — o candidato urgente vence (HTTP 200), o outro perde (HTTP 409) mesmo estando mais próximo da vaga               |
| Dois candidatos não urgentes concorrem pela mesma vaga                      | Sem urgência envolvida, o segundo critério (proximidade) decide o empate?                                                             | <b>Sim</b> — vence quem tinha a consulta mais próxima da vaga liberada                                                       |
| Dois aceites em vagas diferentes chegam ao mesmo tempo                      | O sistema evita falsos empates entre pacientes que na verdade disputam vagas diferentes?                                              | <b>Sim</b> — cada aceite é resolvido de forma independente, ambos os candidatos recebem sua própria vaga (HTTP 200 nos dois) |

**Resultado do arquivo: 3/3 aprovados.** A prioridade “urgência > proximidade” descrita no artigo se confirma mesmo com duas requisições concorrentes reais disputando o mesmo recurso — não é apenas um comportamento correto na leitura do código, mas também sob execução simultânea real, no processo do servidor.

## E.6 Segurança — autenticação, RBAC, injeção de SQL e os dois achados ( security.test.js )

O artigo (Seção IV-A) faz três afirmações centrais de segurança que estes 13 testes verificam: (1) rotas sensíveis exigem token válido; (2) “a separação clara de permissões por papel é considerada essencial (...) para a proteção dos dados do paciente, já que cada usuário acessa apenas as informações pertinentes à sua função”; e (3) “as consultas ao banco utilizam *prepared statements* (...), prevenindo ataques de injeção de SQL”.

### Autenticação (4 testes)

| Teste                             | Pergunta que responde                                                            | Resultado obtido                                                                                                           |
|-----------------------------------|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| GET /reservas sem token           | Uma rota que expõe dados de agenda pode ser acessada sem login?                  | <b>Não</b> — HTTP 401                                                                                                      |
| Token assinado com segredo errado | Um token forjado por um atacante que não conhece o segredo do servidor é aceito? | <b>Não</b> — HTTP 401                                                                                                      |
| Token expirado                    | Uma sessão vencida continua servindo?                                            | <b>Não</b> — HTTP 401                                                                                                      |
| Token válido                      | O caminho positivo (uso legítimo) funciona?                                      | <b>Sim</b> — HTTP 200, confirmando que o bloqueio dos casos anteriores é específico da falha testada, não um erro genérico |

### Controle de acesso por dono do recurso — RBAC (2 testes)

| Teste                                                                     | Pergunta que responde                                                                | Resultado obtido                                                                                                                                             |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Paciente tenta excluir reserva que não é dele nem em que é o profissional | A exclusão de reserva confere se quem pede é o dono ou o profissional responsável?   | <b>Sim</b> — HTTP 403, “não tem permissão”                                                                                                                   |
| Paciente tenta confirmar presença na reserva de outro paciente            | A confirmação de presença está amarrada ao dono da reserva no nível da consulta SQL? | <b>Sim</b> — a query já filtra por <code>usuario_id</code> ; como não há linha correspondente, a atualização afeta zero registros e a rota responde HTTP 404 |

### Injeção de SQL (2 testes)

| Teste                                                                        | Pergunta que responde                                                                                | Resultado obtido                                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Payload <code>' OR '1'='1'; DROP TABLE usuario; --</code> no e-mail do login | O texto malicioso é concatenado à query, ou tratado como um dado comum?                              | <b>Tratado como dado</b> — a query enviada ao driver mysql2 preserva o <code>?</code> (placeholder); o teste captura os parâmetros reais enviados e confirma que o payload chega isolado, como um único parâmetro de <i>bind</i> , sem alterar a estrutura da consulta. A tentativa de login falha normalmente (HTTP 400, “Usuário não encontrado”) |
| O mesmo payload no campo de CPF do cadastro                                  | A validação de negócio (checksum de CPF) já barra a tentativa antes mesmo de o banco ser consultado? | <b>Sim</b> — HTTP 400, e o teste confirma que nenhuma consulta ao banco chegou a ser disparada ( <code>dbRouter.dbPromiseQuery</code> não foi chamada)                                                                                                                                                                                              |

### Achado 1 (encontrado e corrigido durante este experimento) — IDOR nas rotas de perfil de usuário (4 testes)

Ao montar os testes de RBAC, testei também as rotas de perfil de usuário (`backend/src/views/usuariosView.js`) partindo da mesma pergunta feita para reservas: “um usuário autenticado consegue agir sobre o recurso de outro usuário?” A resposta inicial, diferente das rotas de reserva, foi que **sim, conseguia**: nenhuma das rotas conferia se o `:id` da URL correspondia a quem estava logado, permitindo a qualquer paciente ler e editar o perfil de outra pessoa (um IDOR — *Insecure Direct Object Reference*). A falha foi corrigida durante a elaboração deste experimento, adicionando a checagem de posse nas rotas de escrita e, na rota de leitura, restringindo dados de paciente ao próprio dono (o perfil de um profissional continua acessível a qualquer autenticado, pois já é público em `/profissionais` e é assim que o paciente consulta o profissional antes de agendar). Os quatro testes abaixo verificam o comportamento **já corrigido**:

| Teste                                                                                                   | Pergunta que responde                                                                           | Resultado obtido                                                                                                                                   |
|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Usuário autenticado (id 1) chama <code>GET /usuarios/solicitarDados/2</code> , onde 2 é outro paciente  | Um paciente logado ainda consegue ler o perfil completo de outro paciente trocando o id na URL? | <b>Não</b> — HTTP 403. A rota agora verifica o tipo do usuário-alvo antes de decidir                                                               |
| Usuário autenticado (id 1) chama <code>GET /usuarios/solicitarDados/2</code> , onde 2 é um profissional | A correção não quebrou o caso legítimo de ver o perfil de um profissional antes de agendar?     | <b>Não quebrou</b> — HTTP 200 com os dados do profissional; o campo interno <code>tipoUsuario</code> usado só para a checagem não vaza na resposta |
| Usuário autenticado (id 1) chama <code>PATCH /usuarios/2/perfil</code>                                  | Um paciente logado ainda consegue editar o perfil de outra pessoa?                              | <b>Não</b> — HTTP 403, e nenhuma consulta de <code>UPDATE</code> chega a ser disparada no banco                                                    |
| Usuário autenticado (id 1) chama <code>PATCH /usuarios/1/perfil</code> (o próprio perfil)               | A correção preserva o caso de uso normal — editar o próprio perfil?                             | <b>Sim</b> — HTTP 200, atualização aplicada normalmente                                                                                            |

### Achado 2 (encontrado e corrigido) — rota pública exigia token por engano (1 teste)



O artigo declara que “rotas públicas, como login, registro, listagem de profissionais e recuperação de senha, não passam por esse middleware [de autenticação]”. Testei essa afirmação na rota que o próprio front-end trata como pública e encontrei uma divergência: `GET /profissionais` retornava HTTP 401 mesmo sem exigir isso pela regra de negócio. A causa era de roteamento, não de lógica de autorização: `reservasView.js`, `formulariosView.js`, `usuariosView.js` e `vagasView.js` aplicam `router.use(authenticate)` sem restrição de caminho, e no Express isso intercepta **qualquer** requisição que passe por aquele roteador — mesmo sem rota correspondente nele. Como esses roteadores eram montados em `app.js` antes de `profissionaisView` / `empresasView`, todo caminho ainda não respondido por `authView` caía nessa barreira antes de alcançar a rota realmente pública. Corrigido reordenando a montagem dos roteadores em `app.js`, colocando os três roteadores sem autenticação (`uploadsView`, `profissionaisView`, `empresasView`) antes de qualquer roteador com `router.use(authenticate)` global — sem alterar a lógica de autorização em si, já que a rota nunca deveria exigir token.

| Teste                                     | Pergunta que responde                                                                               | Resultado obtido                                                                                          |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>GET /profissionais</code> sem token | A busca pública de profissionais funciona sem login, como um visitante faria antes de se cadastrar? | <b>Sim, agora funciona</b> — HTTP 200 com a lista de profissionais, sem exigir <code>Authorization</code> |

**Resultado do arquivo: 13/13 aprovados**, sendo 8 confirmando que os mecanismos de segurança descritos no artigo funcionam como projetado, 4 confirmando a correção do IDOR de perfil de usuário, e 1 confirmando a correção do roteamento (detalhada no item F).

*Nota: uma rota adicional, `POST /vagas/aceitar/:id` (usada para aceitar uma vaga liberada), também foi cogitada como um possível caso de rota pública mal protegida, por ser mencionada num link de e-mail. Ao checar o front-end, esse link aponta para a tela “Minhas Consultas” sem que o parâmetro do e-mail seja lido automaticamente — o botão que de fato aceita a vaga só aparece depois que o paciente já está logado e navegando na tela. Ou seja, essa rota exigir token é o comportamento correto, e não compõe os achados desta seção.*

## E.7 Rotinas de notificação em segundo plano (unitário) — `notificacoes.jobs.test.js`

O artigo (Seção IV-B) descreve três rotinas assíncronas: o lembrete de confirmação de presença — hoje enviado em cinco marcos fixos antes da consulta (48h, 36h, 24h, 18h e 16h, cada um disparado uma única vez), sendo o de 16h um último aviso destacado por faltar apenas 1h para a liberação automática —, a própria auto-liberação do horário quando o paciente não confirma até 15h antes, e o lembrete horário de urgência sem resposta. Estes 9 testes verificam cada rotina isoladamente, inclusive sua resiliência a falhas parciais (por exemplo, o provedor de e-mail fora do ar).

| Teste                                                                              | Pergunta que responde                                                                                                                                  | Resultado obtido                                                                                                                                                                                       |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserva entra no marco de 24h antes da consulta, sem lembrete de 24h enviado ainda | Cada um dos cinco marcos (48h, 36h, 24h, 18h, 16h) dispara o e-mail e a notificação interna corretos, de forma independente e uma única vez por marco? | <b>Sim</b> — testado no marco de 24h: e-mail enviado ao paciente com data/horário formatados e o número de horas restantes, notificação interna criada, reserva marcada como “lembrete de 24h enviado” |

| Teste                                           | Pergunta que responde                                                                                                                                                                                          | Resultado obtido                                                                                                                                                                                                        |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Reserva entra no marco de 16h antes da consulta | O último aviso (16h) é sinalizado como diferente dos demais, já que a partir dele resta só 1h para a liberação automática (que ocorre às 15h)?                                                                 | <b>Sim</b> — o e-mail troca de assunto (“Última chance — confirme sua presença”) e ganha um destaque visual extra avisando que falta 1h para a liberação automática; a reserva é marcada como “lembrete de 16h enviado” |
| O envio do e-mail de lembrete falha             | Uma falha no provedor de e-mail impede que o restante da rotina (notificação interna, marcação de envio) aconteça?                                                                                             | <b>Não</b> — o job captura o erro, registra em log e continua normalmente; a notificação interna e a marcação são concluídas de qualquer forma                                                                          |
| Nenhuma reserva em nenhum dos cinco marcos      | O job evita disparos desnecessários quando não há nada a fazer?                                                                                                                                                | <b>Sim</b> — nenhuma chamada de e-mail ou notificação é feita                                                                                                                                                           |
| Reserva chega a 15h da consulta sem confirmação | A auto-liberação realmente libera o horário e avisa as duas partes (paciente e profissional)?                                                                                                                  | <b>Sim</b> — horário liberado, e-mail ao paciente, notificação ao profissional                                                                                                                                          |
| A liberação no banco falha para uma reserva     | O sistema evita avisar paciente e profissional sobre uma liberação que na verdade não ocorreu?                                                                                                                 | <b>Sim</b> — se o <code>UPDATE</code> falha, nem o e-mail nem a notificação são disparados para aquela reserva específica                                                                                               |
| Nenhuma reserva elegível para auto-liberação    | O job também evita ações desnecessárias nesse segundo cenário?                                                                                                                                                 | <b>Sim</b> — nenhuma liberação é executada                                                                                                                                                                              |
| Urgência pendente há mais de 1h sem resposta    | O lembrete horário de urgência avisa tanto o profissional quanto o paciente, como o artigo descreve (“lembrete por e-mail ao profissional (...) e uma notificação (...) O mesmo ciclo se repete a cada hora”)? | <b>Sim</b> — dois e-mails, duas notificações internas e a marcação de lembrete enviado, todos disparados numa única passada do job                                                                                      |
| Nenhuma urgência pendente há mais de 1h         | O job fica ocioso quando não há nada pendente?                                                                                                                                                                 | <b>Sim</b> — nenhuma chamada é feita                                                                                                                                                                                    |

**Resultado do arquivo: 9/9 aprovados.** As três rotinas assíncronas funcionam como descrito no artigo — a de confirmação de presença hoje com um cronograma mais granular de cinco avisos, em vez de um único disparo ao entrar na janela de 48h — e continuam operando de forma consistente mesmo diante de falhas parciais de um serviço externo (e-mail).

## E.8 Auditoria ampliada de IDOR (segurança) — `idor-ampliado.test.js`

O achado do item E.6 (IDOR em `usuariosView.js`) levantou uma pergunta natural: existe o mesmo tipo de falha em outras partes do sistema? Uma segunda rodada de revisão, percorrendo cada rota de `reservasView.js`, `formulariosView.js`, `vagasView.js` e `avaliacoesView.js`, encontrou o mesmo padrão — a rota exige um token válido, mas não confere se o dado pedido pertence a quem está pedindo — repetido em **seis pontos adicionais**, alguns mais graves que o original por envolverem dados clínicos. Todos foram corrigidos; os 20 testes abaixo comprovam a correção.

| Teste                                                                        | Pergunta que responde                                                                                                                                                               | Resultado obtido                                                                                                                                                                                                                                                                                   |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET /reservas?profissional_id=X feita por um paciente                        | Um paciente autenticado consegue listar as consultas de <b>todos os pacientes</b> de um profissional só informando o id dele na query string?                                       | <b>Não mais</b> — a rota agora ignora por completo os parâmetros da query string e usa exclusivamente o req.userId do token para decidir o filtro. Antes da correção, isso vazava nome, telefone, e-mail, descrição de urgência e o arquivo anexado de cada paciente de um profissional arbitrário |
| GET /reservas feita por um profissional                                      | A correção acima preserva o uso legítimo — o profissional ver a própria agenda?                                                                                                     | <b>Sim</b> — continua funcionando normalmente                                                                                                                                                                                                                                                      |
| GET /reservas?usuario_id=X feita por um profissional logado em modo paciente | Um profissional que também usa o sistema como paciente consegue pedir explicitamente só as próprias reservas como paciente, via usuario_id ?                                        | <b>Sim</b> — continua funcionando, sem reabrir a brecha: o filtro é sempre amarrado ao req.userId, então só é possível pedir as próprias reservas, nunca as de outra pessoa                                                                                                                        |
| GET /formularios/reserva/:id por alguém alheio à consulta                    | O formulário de pré-consulta (sintomas, histórico de saúde, medicamentos — dado sensível pela LGPD) pode ser lido por qualquer autenticado, bastando adivinhar o número da reserva? | <b>Não mais</b> — HTTP 403 para quem não é nem o paciente nem o profissional daquela consulta específica                                                                                                                                                                                           |
| GET /formularios/reserva/:id pelo paciente dono da consulta                  | A correção preserva o acesso do próprio paciente ao seu formulário?                                                                                                                 | <b>Sim</b>                                                                                                                                                                                                                                                                                         |
| GET /formularios/reserva/:id pelo profissional responsável                   | E o acesso do profissional que vai atender essa consulta?                                                                                                                           | <b>Sim</b>                                                                                                                                                                                                                                                                                         |
| POST /formularios informando o usuarioId de outra pessoa                     | É possível gravar respostas de anamnese em nome de outro paciente?                                                                                                                  | <b>Não mais</b> — HTTP 403 se o usuarioId enviado não for o do próprio token                                                                                                                                                                                                                       |
| GET /vagas/candidatos?profissional_id=X por outro profissional               | Um profissional consegue ver a lista de candidatos (nomes, e-mails, urgências) da agenda de outro profissional?                                                                     | <b>Não mais</b> — HTTP 403                                                                                                                                                                                                                                                                         |
| POST /vagas/notificar em nome de outro profissional                          | É possível disparar notificações de vaga fingindo ser outro profissional?                                                                                                           | <b>Não mais</b> — HTTP 403                                                                                                                                                                                                                                                                         |
| GET /vagas/pendentes/:usuarioId de outro paciente                            | Um paciente vê as vagas pendentes oferecidas a outro paciente?                                                                                                                      | <b>Não mais</b> — HTTP 403                                                                                                                                                                                                                                                                         |
| POST /vagas/aceitar com token e posse corretos                               | A correção não quebrou o fluxo normal de aceite de vaga?                                                                                                                            | <b>Não quebrou</b> — continua funcionando (o teste chega até a etapa de checar consultas disponíveis)                                                                                                                                                                                              |
| POST /vagas/aceitar com token de uma notificação, mas de outro usuário       | Mesmo sabendo o token secreto de uma notificação, um usuário autenticado como outra pessoa consegue aceitá-la?                                                                      | <b>Não</b> — HTTP 403; a posse do token sozinha não basta mais, o usuário autenticado precisa ser o notificado                                                                                                                                                                                     |

| Teste                                                              | Pergunta que responde                                                                                                                        | Resultado obtido                                                                                                        |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| POST /vagas/recusar de uma notificação de outro paciente           | Esta era a falha mais simples de explorar: a rota não pedia nada além do id na URL para recusar a vaga de qualquer um. Continua sendo assim? | <b>Não mais</b> — HTTP 403                                                                                              |
| POST /vagas/recusar da própria notificação, sem enviar token       | A correção respeita o fato de o front-end nunca ter enviado um token nessa rota?                                                             | <b>Sim</b> — a posse agora é conferida pelo usuário autenticado (JWT), sem exigir um token que a interface não envia    |
| GET /notificacoes-profissional/:id de outro profissional           | Um profissional lê a caixa de notificações internas de outro?                                                                                | <b>Não mais</b> — HTTP 403                                                                                              |
| GET /vagas/notificados-pendentes/:id de outro profissional         | Mesma pergunta, para a lista de pacientes já notificados de uma vaga?                                                                        | <b>Não mais</b> — HTTP 403                                                                                              |
| GET /notificacoes-paciente/:usuarioId de outro paciente            | Um paciente lê as notificações internas de outro?                                                                                            | <b>Não mais</b> — HTTP 403                                                                                              |
| GET /notificacoes-paciente/:usuarioId do próprio paciente          | Acesso às próprias notificações continua funcionando?                                                                                        | <b>Sim</b>                                                                                                              |
| POST /avaliacoes informando um usuario_id de outra pessoa no corpo | É possível publicar uma avaliação de profissional se passando por outro paciente?                                                            | <b>Não mais</b> — o usuario_id é sempre atribuído a partir do token ( req.userId ), o valor enviado no corpo é ignorado |
| POST /avaliacoes sem token                                         | A rota de avaliação, que antes não exigia autenticação nenhuma, agora exige?                                                                 | <b>Sim</b> — HTTP 401                                                                                                   |

**Resultado do arquivo: 19/19 aprovados.** Antes de cada correção, o código-fonte do front-end ( fisiopilattes\_2/src ) foi conferido para garantir que nenhuma tela legítima dependia de acessar dados de outra pessoa — todos os usos reais desses endpoints já passavam o próprio id do usuário logado — de forma que nenhuma correção quebra uma funcionalidade existente.

## E.9 Download de arquivos enviados — exames e documentos de urgência (segurança) — uploads.security.test.js

As duas rodadas anteriores de auditoria (E.6, E.8) cobriram as rotas que devolvem *metadados* protegidos por dono. Uma pergunta ficou de fora: os *arquivos* em si — o anexo de exame enviado num formulário de pré-consulta ( exame\_anexo ) e o documento anexado numa urgência ( arquivo\_urgencia ) — estão protegidos pelo mesmo critério, ou só o link para eles está? A resposta encontrada foi que **só o link estava protegido**: o arquivo em disco era servido por `app.use('/uploads', express.static(...))`, uma rota sem nenhuma autenticação, bastando conhecer (ou adivinhar) o nome do arquivo salvo — que seguia o padrão previsível <timestamp>-<nome original>. Qualquer checagem de posse feita nas rotas de `formulariosView / reservasView` (E.6, E.8) era inútil contra alguém que já possuísse ou adivinhasse a URL

direta do arquivo. Por se tratar de dado de saúde (exame anexado), essa é a falha mais sensível encontrada em toda a auditoria do ponto de vista da LGPD.

| Teste                                                                                     | Pergunta que responde                                                                                                                                                                              | Resultado obtido                                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET /uploads/<arquivo> sem token                                                          | O arquivo salvo em disco pode ser baixado diretamente, sem autenticação, bastando saber o nome dele?                                                                                               | <b>Não mais</b> — HTTP 401                                                                                                                                                                                                                                             |
| Token assinado para outro arquivo                                                         | Um token válido, mas emitido para um arquivo diferente, dá acesso a este?                                                                                                                          | <b>Não</b> — HTTP 403; o token é amarrado ao nome do arquivo                                                                                                                                                                                                           |
| Token expirado                                                                            | Um link antigo continua funcionando indefinidamente?                                                                                                                                               | <b>Não</b> — HTTP 401; o token expira em 5 minutos, no mesmo padrão já usado para o armazenamento em S3                                                                                                                                                                |
| Token assinado com segredo errado                                                         | Um token forjado por quem não conhece o segredo do servidor é aceito?                                                                                                                              | <b>Não</b> — HTTP 401                                                                                                                                                                                                                                                  |
| Nome de arquivo com tentativa de <i>path traversal</i> ( <code>../..package.json</code> ) | É possível usar a rota de download para ler arquivos fora da pasta de uploads?                                                                                                                     | <b>Não</b> — a rota rejeita nomes de arquivo que não correspondem exatamente ao <code>path.basename</code> esperado                                                                                                                                                    |
| Token válido, correspondente ao arquivo pedido                                            | O caso de uso legítimo continua funcionando?                                                                                                                                                       | <b>Sim</b> — HTTP 200, conteúdo do arquivo devolvido corretamente                                                                                                                                                                                                      |
| <code>getFileUrl</code> para armazenamento local                                          | A função que gera a URL devolvida ao front-end ( <code>formulariosView</code> , <code>reservasView</code> ) já embute esse token automaticamente, sem exigir mudança nas telas que exibem o anexo? | <b>Sim</b> — a URL relativa devolvida já vem no formato <code>/uploads/&lt;arquivo&gt;?token=&lt;jwt&gt;</code> , consumida do mesmo jeito que antes por um <code>&lt;a href&gt;</code> simples no front-end, sem precisar enviar cabeçalho <code>Authorization</code> |

**Resultado do arquivo: 7/7 aprovados.** Corrigido substituindo o `express.static` público por uma rota ( `backend/src/views/uploadsView.js` ) que exige um token JWT de curta duração (5 min), amarrado ao nome do arquivo e emitido apenas depois que a checagem de posse do dado (já existente em `formulariosView / reservasView` ) passou — o mesmo modelo de segurança que já era usado para o armazenamento em S3 (URL pré-assinada com expiração de 300s), agora replicado também para o armazenamento local. Nenhuma mudança foi necessária no front-end: como o anexo já era exibido como um link simples ( `<a href>` ), e não via uma chamada autenticada via JavaScript, a solução precisava funcionar sem cabeçalho `Authorization` — daí o token ir embutido na própria URL, e não num header.

## E.10 Auditoria por classes de vulnerabilidade e direito de exclusão de conta (segurança)

— `seguranca-ampliada2.test.js`

As três rodadas anteriores (E.6, E.8, E.9) percorreram rotas específicas em busca do mesmo tipo de falha (IDOR). Esta quarta rodada mudou de método: em vez de auditar rota por rota, revisei o sistema por

**classes** de vulnerabilidade reconhecidas (OWASP e LGPD) — força bruta, gestão de segredo de sessão, exposição de dado por enumeração, cabeçalhos de segurança HTTP e direitos do titular dos dados — percorrendo `config.js`, `middlewares/`, `authView.js` e os modelos, não só as rotas de `views/`.

| Teste                                                                | Pergunta que responde                                                                                                                                                                                                                             | Resultado obtido                                                                                                                                                                                                                                                                                                                                                                                   |
|----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Carregar <code>config.js</code> sem <code>JWT_SECRET</code> definido | O segredo usado para assinar todo token de sessão tem algum valor padrão previsível, caso a variável de ambiente seja esquecida?                                                                                                                  | <b>Não mais</b> — o módulo lança um erro e recusa iniciar. Antes, o valor padrão era a palavra <code>'secreto'</code> , escrita no próprio código-fonte; qualquer pessoa que a conhecesse podia forjar um token JWT válido para <b>qualquer</b> id de usuário, sem nunca ter feito login, e todas as checagens de posse (Achados 1–3) seriam inúteis contra isso                                   |
| GET <code>/user/:id</code> sem token                                 | É possível coletar nome e e-mail de qualquer usuário do sistema testando ids sequenciais (1, 2, 3...), sem autenticação e sem limite de tentativas?                                                                                               | <b>Não mais</b> — HTTP 401. Confirmado no código do front-end que nenhuma tela usa essa rota hoje                                                                                                                                                                                                                                                                                                  |
| GET <code>/user/:id</code> com token válido                          | A correção não quebrou o caso de uso legítimo (se algum dia essa rota for usada)?                                                                                                                                                                 | <b>Não quebrou</b> — HTTP 200, mesmos dados de antes                                                                                                                                                                                                                                                                                                                                               |
| 11ª tentativa de POST <code>/login</code> no mesmo IP em 15 minutos  | <code>/login</code> , <code>/register</code> e <code>/api/forgot-password</code> aceitam tentativas ilimitadas de senha contra um e-mail conhecido?                                                                                               | <b>Não mais</b> — HTTP 429 a partir da 11ª tentativa. Antes não havia nenhum limite, atraso ou bloqueio                                                                                                                                                                                                                                                                                            |
| DELETE <code>/usuarios/:id</code> de outro usuário                   | Um usuário autenticado consegue excluir a conta de outra pessoa?                                                                                                                                                                                  | <b>Não</b> — HTTP 403                                                                                                                                                                                                                                                                                                                                                                              |
| DELETE <code>/usuarios/:id</code> sem token                          | A exclusão de conta está protegida por autenticação?                                                                                                                                                                                              | <b>Sim</b> — HTTP 401                                                                                                                                                                                                                                                                                                                                                                              |
| DELETE <code>/usuarios/:id</code> da própria conta                   | O botão “Excluir conta” (novo, em <code>/Conta</code> ) realmente remove os dados associados — inclusive nas três tabelas de notificação que não têm FOREIGN KEY para <code>usuario</code> e por isso não seriam limpas por uma cascata do banco? | <b>Sim</b> — o teste confirma que <code>notificacoes_vaga</code> , <code>notificacoes_profissional</code> e <code>notificacoes_paciente</code> recebem DELETE explícito para o id do usuário, além da exclusão da própria linha em <code>usuario</code> (que dispara em cascata <code>reservas</code> , <code>reset_tokens</code> e <code>avaliacoes</code> via FOREIGN KEY ... ON DELETE CASCADE) |
| DELETE <code>/usuarios/:id</code> de um id inexistente               | A rota distingue “excluí com sucesso” de “esse usuário nunca existiu”?                                                                                                                                                                            | <b>Sim</b> — HTTP 404 em vez de 200                                                                                                                                                                                                                                                                                                                                                                |

**Resultado do arquivo: 8/8 aprovados.** Corrigido: (1) segredo do JWT sem valor padrão inseguro; (2) rate limiting em `/login` (10/15min), `/register` (15/hora) e `/api/forgot-password` (5/15min), via `express-`

`rate-limit`; (3) `GET /user/:id` agora autenticado; (4) cabeçalhos de segurança HTTP padrão via `helmet`; (5) nova rota `DELETE /usuarios/:id`, restrita ao próprio dono, implementando o direito de exclusão da LGPD, com botão correspondente em “Minha Conta”. Dois pontos ficaram fora do escopo desta rodada, registrados como não corrigidos: ausência de log/trilha de auditoria de acesso, e o valor do segredo já configurado no `.env` deste ambiente (que continuava sendo o mesmo `'secreto'` que antes era só o padrão do código). Ambos foram fechados em seguida — ver seção E.11 e a atualização do Achado 4.

## E.11 Log de auditoria de acesso (segurança) — `auditoria.test.js`

O Achado 4 (E.10) deixou registrado como pendente, deliberadamente fora daquela rodada, a ausência de qualquer log de quem fez o quê no sistema. Sem isso, em caso de incidente de segurança não haveria como reconstruir a sequência de eventos — nem mesmo saber se uma conta específica sofreu tentativas de login por força bruta antes de o rate limiting existir. Esta rodada fecha esse ponto.

| Teste                                              | Pergunta que responde                                                                                                                                     | Resultado obtido                                                                                                                                                                                                                                                                |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Login com senha correta                            | Um login bem-sucedido fica registrado, com o id do usuário, no log de auditoria?                                                                          | <b>Sim</b> — linha inserida em <code>log_acesso</code> com <code>evento='login'</code> , <code>sucesso=1</code> , IP de origem e o id do usuário                                                                                                                                |
| Login com senha errada                             | Uma tentativa de login que falha por senha incorreta também é registrada, e diferenciada de um sucesso?                                                   | <b>Sim</b> — <code>sucesso=0</code> , com o id do usuário (a senha estava errada, mas o e-mail existe) e o motivo em <code>detalhe</code>                                                                                                                                       |
| Login com e-mail inexistente                       | O sistema registra tentativas contra e-mails que nem existem na base (útil para detectar varredura/enumeração), mesmo sem um id de usuário para associar? | <b>Sim</b> — <code>sucesso=0</code> , <code>usuario_id=NULL</code> , e o e-mail tentado fica no campo <code>detalhe</code>                                                                                                                                                      |
| <code>DELETE /usuarios/:id</code> da própria conta | A ação mais irreversível do sistema (exclusão de conta) fica registrada antes de o vestígio (a própria linha do usuário) desaparecer?                     | <b>Sim</b> — o evento é gravado com o id do usuário <i>antes</i> da exclusão; o <code>detalhe</code> também guarda o id em texto, então o registro do evento sobrevive mesmo depois que a <code>FOREIGN KEY ... ON DELETE SET NULL</code> zera a coluna <code>usuario_id</code> |
| Falha ao gravar o log (tabela indisponível)        | Uma falha na gravação do log de auditoria consegue derrubar a operação principal (ex.: impedir um login válido)?                                          | <b>Não</b> — a escrita do log é protegida por seu próprio <code>try/catch</code> e não é aguardada ( <i>fire-and-forget</i> ); o login segue respondendo normalmente mesmo com a inserção do log falhando                                                                       |

**Resultado do arquivo: 5/5 aprovados.** Implementado: nova tabela `log_acesso` (migração `20260805_create_log_acesso.sql`, aplicada automaticamente na subida do servidor, como as demais migrações do projeto) e `models/auditModel.js`, registrando login (sucesso e falha), cadastro, redefinição de senha concluída e exclusão de conta, cada um com id do usuário (quando conhecido), IP de origem e

timestamp. Validado manualmente contra o MySQL real (seção G.2): registro, tentativa de login com senha errada e login correto de um usuário real geraram exatamente as três linhas esperadas na tabela.

## F. Achados e discussão

Do total de 97 testes, **todos confirmam que a implementação corresponde ao que o artigo descreve ou já foi corrigida para corresponder** — não há, neste momento, nenhum achado desta auditoria conhecido e ainda pendente de correção, nem no código nem na configuração do ambiente. Os quatro achados abaixo foram encontrados e corrigidos ao longo do experimento, em cinco rodadas sucessivas de investigação, cada uma motivada por uma pergunta direta sobre o quão seguro o sistema realmente estava.

### Achado 1 (corrigido) — padrão de IDOR repetido em sete rotas do backend

A investigação começou pontual, em `backend/src/views/usuariosView.js`: as rotas de perfil exigiam apenas `router.use(authenticate)` (um token válido qualquer), sem conferir se o `:id` da URL era o mesmo usuário do token. Isso permitia que qualquer paciente ou profissional autenticado lesse e editasse o perfil de qualquer outro usuário. Ao ser questionado diretamente se esse era o único problema de segurança do sistema, o mesmo padrão foi buscado sistematicamente em todas as demais rotas autenticadas — e encontrado em mais **seis pontos**, listados da mais para a menos grave:

| Rota                                                                                                                                                                          | O que vazava/permitia antes da correção                                                                                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>GET /reservas?profissional_id=X</code>                                                                                                                                  | Listava as consultas de <b>todos os pacientes</b> de qualquer profissional — nome, telefone, e-mail, descrição de urgência e arquivo anexado — para qualquer paciente autenticado |
| <code>GET /formularios/reserva/:id</code>                                                                                                                                     | Expunha o formulário de pré-consulta (sintomas, histórico de saúde, medicamentos — dado sensível pela LGPD) de qualquer consulta a qualquer autenticado                           |
| <code>GET/PATCH /usuarios/.../perfil,localizacao,informacoes,solicitarDados</code>                                                                                            | Leitura e edição do perfil de qualquer outro usuário (achado original)                                                                                                            |
| <code>GET /vagas/candidatos</code> , <code>POST /vagas/notificar</code> , <code>GET /notificacoes-profissional/:id</code> , <code>GET /vagas/notificados-pendentes/:id</code> | Permitiam ver/agir sobre a agenda e as notificações internas de <b>outro profissional</b>                                                                                         |
| <code>POST /vagas/recusar/:id</code>                                                                                                                                          | Recusava a vaga de qualquer paciente sem exigir absolutamente nenhuma prova de posse (nem token, nem checagem de usuário)                                                         |
| <code>GET /vagas/pendentes/:usuarioId</code> , <code>GET/POST /notificacoes-paciente/:usuarioId</code>                                                                        | Liam/marcavam como lidas as notificações internas de <b>outro paciente</b>                                                                                                        |
| <code>POST /avaliacoes</code>                                                                                                                                                 | Publicava uma avaliação em nome de qualquer paciente, bastando informar o <code>usuario_id</code> de outra pessoa no corpo da requisição                                          |



**Correção aplicada em todos os casos:** cada rota agora compara o identificador do recurso pedido com `req.userId` (extraído do JWT, portanto não falsificável pelo cliente) e responde HTTP 403 quando não coincidem — o mesmo padrão que já existia corretamente em `reservasView.js` para exclusão de reservas, agora replicado de forma consistente. Duas exceções deliberadas, ambas confirmadas no código-fonte do front-end antes da correção: (1) o perfil de um usuário do tipo **profissional** continua legível por qualquer autenticado, pois é a mesma informação já pública em `/profissionais`, e é assim que o paciente vê o profissional antes de agendar; (2) `POST /vagas/recusar` passou a exigir apenas o usuário autenticado (não um token adicional), porque o front-end nunca enviou esse token nessa chamada — exigir-lo quebraria a funcionalidade real. Os 20 testes da seção E.8 comprovam a correção, e a suíte completa permanece 100% aprovada após todas as mudanças.

## Achado 2 (corrigido) — rota pública exigindo token por erro de montagem do roteador

Como descrito no item E.6, `GET /profissionais` deveria ser acessível sem autenticação — o próprio front-end (`EmpresasProfissionais`) já pressupõe isso, carregando a lista de profissionais independentemente de haver um usuário logado — mas exigia um token válido por causa da ordem de montagem dos roteadores em `app.js` combinada com o `router.use(authenticate)` sem escopo de caminho em `reservasView.js`, `formulariosView.js`, `usuariosView.js` e `vagasView.js`. Esse padrão — middleware de autenticação aplicado a um roteador inteiro, sem restrição de caminho — intercepta qualquer requisição que passe por ali, mesmo sem rota correspondente; como esses roteadores eram montados antes dos roteadores públicos, uma requisição a `/profissionais` (ou a `/empresas`) nunca chegava a sair do primeiro roteador autenticado do caminho.

**Correção aplicada:** os três roteadores sem autenticação do sistema (`uploadsView`, `profissionaisView`, `empresasView`) foram reordenados em `app.js` para serem montados antes de qualquer roteador com `router.use(authenticate)` global — sem alterar nenhuma regra de autorização, já que essas rotas nunca deveriam exigir token. O teste da seção E.6 comprova o novo comportamento (HTTP 200 sem `Authorization`).

Uma segunda rota, `POST /vagas/aceitar/:id`, chegou a ser cogitada como parte do mesmo achado por aparecer referenciada num link de e-mail, mas a investigação do fluxo real no front-end mostrou que essa suspeita não se confirma: o link do e-mail leva a um parâmetro de URL que a tela “Minhas Consultas” não lê automaticamente, e o botão que efetivamente aceita a vaga só existe depois que o paciente já está autenticado e navegando na tela. Portanto essa rota exigir token está correto, e não é tratada como achado.

## Achado 3 (corrigido) — arquivos enviados (exames, documentos de urgência) eram servidos publicamente

O achado mais sensível da auditoria do ponto de vista da LGPD. As correções dos Achados 1 e 2, e da seção E.8, protegem os *metadados* de uma consulta ou formulário (quem pode ler o registro no banco), mas nenhuma delas alcança o *arquivo em si* quando o sistema roda em armazenamento local (sem S3 configurado): `backend/src/app.js` expunha a pasta inteira de uploads via `express.static('/uploads', ...)`, sem qualquer autenticação. O nome de cada arquivo seguia o padrão previsível `<timestamp-em-milissegundos>-<nome-original-sanitizado>`, de forma que qualquer pessoa que descobrisse ou adivinhasse um nome de arquivo — inclusive um anexo de exame (`exame_anexo`, no formulário de pré-

consulta) — conseguia baixá-lo diretamente, para sempre, sem estar logada e sem ser a paciente ou o profissional daquela consulta. Verificado na pasta `backend/uploads/` deste próprio ambiente de desenvolvimento: havia arquivos reais de uso anterior do sistema acessíveis exatamente dessa forma.

**Correção aplicada:** detalhada na seção E.9 — a rota pública de arquivos estáticos foi substituída por uma rota que exige um token assinado (JWT, 5 min de validade) amarrado ao nome do arquivo, gerado apenas depois que a checagem de posse já feita em `formulariosView/ reservasView` (Achado 1, seção E.8) é aprovada. O mesmo modelo de segurança já existia para arquivos armazenados em S3 (URL pré-assinada de 300s); a correção o estende também ao armazenamento local, sem exigir nenhuma mudança no front-end. Os 7 testes de `uploads.security.test.js` comprovam a correção.

**Nota sobre S3:** no ambiente de produção configurado neste projeto ( `.env` com `S3_ENDPOINT / S3_BUCKET` preenchidos), o armazenamento local nunca chega a ser usado — todo upload novo vai para o S3/MinIO, já protegido por URL pré-assinada. Ainda assim, a correção era necessária: o armazenamento local é o *fallback* automático e silencioso caso as variáveis de ambiente do S3 fiquem ausentes (erro de configuração, migração de servidor, ambiente de homologação), e sem ela um erro de configuração transformaria silenciosamente todos os exames anexados em dados públicos.

## Achado 4 (corrigido) — segredo do JWT sem valor padrão seguro, força bruta sem limite e exposição de dado por enumeração

A quarta rodada de investigação (E.10) trocou o método: em vez de auditar rota por rota, revisei classes inteiras de vulnerabilidade reconhecidas — força bruta, segredo de sessão, exposição por enumeração — em `config.js`, `middlewares/` e `authView.js`, não só nas rotas de `views/`. O achado mais grave de toda a auditoria estava aqui: `config.js` continha `jwtSecret: process.env.JWT_SECRET || 'segredo'`. O `JWT_SECRET` é a chave usada pelo servidor para assinar (e depois verificar) todo token de sessão — não é gerado a cada login, é um único valor configurado uma vez para todo o servidor. Um valor padrão escrito no próprio código-fonte, usado sempre que a variável de ambiente não for configurada, não é segredo nenhum: confirmei localmente que, com esse valor, é possível forjar um token JWT válido para **qualquer id de usuário**, sem nunca ter feito login — o que anulava todas as checagens de posse dos Achados 1–3, já que todas elas confiam no `req.userId` extraído do token.

Junto com esse achado, a mesma rodada encontrou: `GET /user/:id` sem autenticação, permitindo coletar nome e e-mail de qualquer usuário testando ids sequenciais (dado pessoal exposto por simples enumeração); e ausência total de limite de tentativas em `/login`, `/register` e `/api/forgot-password`, permitindo força bruta de senha ou varredura de e-mails cadastrados sem qualquer fricção.

**Correção aplicada:** (1) `config.js` agora lança um erro e recusa iniciar o servidor se `JWT_SECRET` não estiver definido, eliminando o valor padrão inseguro; (2) `GET /user/:id` passou a exigir `authenticate` (nenhuma tela do front-end usava essa rota, confirmado por busca no código); (3) `express-rate-limit` aplicado a `/login` (10 tentativas/15min), `/register` (15/hora) e `/api/forgot-password` (5/15min); (4) `helmet` adicionado para os cabeçalhos de segurança HTTP padrão. Os 8 testes de `seguranca-ampliada2.test.js` (seção E.10) comprovam as quatro correções.

**Ressalva fechada numa rodada seguinte:** a correção do `config.js` impedia o servidor de rodar *silenciosamente* com um segredo previsível, mas não trocava, sozinha, o valor que já estava configurado no `.env` deste ambiente — que continuava sendo `segredo`, o mesmo valor que antes era só o padrão do

código. Como trocar o conteúdo de uma variável de ambiente é uma mudança de credencial, e não de código-fonte, isso foi feito à parte, mediante confirmação explícita: o `.env` deste ambiente foi atualizado para um segredo aleatório de 96 caracteres hexadecimais ( `crypto.randomBytes(48).toString('hex')` ), o que também invalidou (como esperado) qualquer sessão ativa assinada com o valor anterior.

**Também identificado nesta rodada e fechado logo em seguida (seção E.11):** ausência de log/trilha de auditoria de acesso — não havia registro de quem acessou o quê, relevante em caso de incidente e para a rastreabilidade de acesso a dado pessoal exigida pela LGPD.

**Nova funcionalidade decorrente da auditoria — exclusão de conta (direito do titular, LGPD)**

Nenhuma rota do sistema implementava a exclusão de conta ou a remoção dos dados associados a um usuário — um direito garantido ao titular dos dados pela LGPD. Foi adicionada a rota `DELETE /usuarios/:id` (restrita ao próprio dono, mesma checagem `exigirDono` já usada nas demais rotas de edição de perfil), que remove a linha do usuário — disparando em cascata, via `FOREIGN KEY ... ON DELETE CASCADE`, a exclusão de `reservas`, `reset_tokens` e `avaliacoes` — e remove manualmente as linhas em `notificacoes_vaga`, `notificacoes_profissional` e `notificacoes_paciente`, que não têm `FOREIGN KEY` para `usuario` e por isso não seriam limpas automaticamente pelo banco. Os arquivos locais de exame/urgência associados às reservas do usuário também são apagados do disco. Um botão “Excluir conta” foi adicionado à tela “Minha Conta”, ao lado de “Sair da conta”, com uma confirmação inline (mensagem de aviso e botão “Sim, excluir minha conta” exibidos abaixo do botão, sem depender de pop-up do navegador) antes de executar, por se tratar de uma ação irreversível. Testado manualmente no navegador pelo próprio autor, confirmando o fluxo completo de ponta a ponta.

**G. Auditoria de dependências e validação manual contra a aplicação real**

Os 97 testes automatizados (itens D–E) rodam com o banco de dados e o provedor de e-mail substituídos por mocks — o que garante determinismo, mas não prova, sozinho, que a aplicação real (com MySQL e armazenamento de arquivos de verdade) se comporta da mesma forma. Dois exames complementares, fora da suíte Jest, foram feitos para cobrir essa lacuna.

**G.1 `npm audit` — vulnerabilidades conhecidas nas dependências**

Nenhum dos testes anteriores audita as bibliotecas de terceiros usadas pelo projeto. Rodar `npm audit` no diretório `backend/` encontrou 5 vulnerabilidades conhecidas (2 moderadas, 3 altas) publicadas para versões instaladas de pacotes transitivos:

| Pacote                                                             | Severidade | Vulnerabilidade                                                                                                                                      |
|--------------------------------------------------------------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>jws</code> (dependência de <code>jsonwebtoken</code> )       | Alta       | <i>Improperly Verifies HMAC Signature</i> (GHSA-869p-cjfg-cm3x) — na própria biblioteca que verifica a assinatura de todo token de sessão do sistema |
| <code>path-to-regexp</code> (dependência de <code>express</code> ) | Alta       | Negação de serviço por expressão regular (ReDoS) ao processar rotas                                                                                  |

| Pacote                                                          | Severidade | Vulnerabilidade                                                                                                                                   |
|-----------------------------------------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                 |            | com múltiplos parâmetros (GHSA-37ch-88jc-xwx2)                                                                                                    |
| <code>body-parser</code> (dependência de <code>express</code> ) | Moderada   | Negação de serviço quando um valor de limite inválido desativa silenciosamente o controle de tamanho do corpo da requisição (GHSA-v422-hmwv-36x6) |
| <code>qs</code>                                                 | Moderada   | Três falhas de negação de serviço na análise de query strings (GHSA-w7fw-mjwx-w883 e relacionadas)                                                |

O achado mais relevante para este trabalho é o de `jws`: por estar na cadeia de verificação de assinatura do JWT, uma falha ali teria o mesmo tipo de impacto do Achado 4 (bypass de autenticação) mesmo com o `JWT_SECRET` corrigido.

**Correção aplicada:** `npm audit fix` atualizou `jws` (3.2.2 → 3.2.3), `express` (4.21.2 → 4.22.2), `path-to-regexp`, `qs` e `body-parser` para as versões corrigidas, sem quebra de compatibilidade (nenhuma versão *major* mudou). Rodar `npm test` após a atualização confirmou 89/89 aprovados, sem regressão. `npm audit` após a correção: **0 vulnerabilidades**.

## G.2 Validação manual contra o backend real (MySQL) e o front-end real

Para confirmar que o comportamento observado nos testes com mock também se sustenta com o banco de dados de verdade, o backend foi executado localmente ( `npm start` ) contra a instância real de MySQL e MinIO do ambiente de desenvolvimento, e testado com requisições HTTP reais ( `curl` ), sem nenhum mock:

| Verificação manual                                                                                                 | Resultado                                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>GET /profissionais</code> sem token                                                                          | HTTP 200 (rota pública funcionando, achado 2)                                                                                                                |
| <code>GET /reservas</code> sem token                                                                               | HTTP 401                                                                                                                                                     |
| Cabeçalhos de resposta do <code>helmet</code>                                                                      | <code>Content-Security-Policy</code> , <code>Strict-Transport-Security</code> , <code>X-Content-Type-Options</code> , <code>X-Frame-Options</code> presentes |
| <code>GET /uploads/&lt;arquivo&gt;</code> sem token                                                                | HTTP 401 (achado 3)                                                                                                                                          |
| 11ª tentativa de login errada em sequência                                                                         | HTTP 429 (rate limiting, achado 4)                                                                                                                           |
| <code>GET /user/:id</code> sem token                                                                               | HTTP 401 (achado 4)                                                                                                                                          |
| Cadastro de um usuário real, login, e <code>DELETE /usuarios/:id</code> de <b>outro</b> id com o próprio token     | HTTP 403                                                                                                                                                     |
| <code>DELETE /usuarios/:id</code> da própria conta, seguido de nova tentativa de login                             | Conta excluída (HTTP 200); login seguinte retornou "Usuário não encontrado", confirmando a remoção real no banco                                             |
| Cadastro real → login com senha errada → login correto, consultando <code>log_acesso</code> direto no MySQL depois | Três linhas gravadas, na ordem certa: <code>cadastro</code> (sucesso=1), <code>login</code> (sucesso=0, detalhe "senha                                       |

| Verificação manual | Resultado                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------|
|                    | incorreta”), login (sucesso=1) — todas com o id correto do usuário e o IP de origem (seção E.11) |

Todas as nove verificações reproduziram exatamente o comportamento que os testes automatizados com mock previam, sem nenhuma divergência. Adicionalmente, `npm run build` do front-end ( `fisiopilattes_2/` ) foi executado sem erros, e o botão “Excluir conta” foi testado manualmente no navegador pelo autor contra o backend real, com resultado positivo.

## H. Como reproduzir

---

```
cd backend
npm install
npm test
```

Nenhuma variável de ambiente de banco de dados ou de provedor de e-mail é necessária — todas as dependências externas são substituídas por mocks determinísticos, como descrito no item B.